

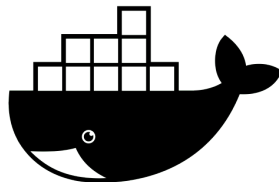
Granja de modelos de IA

Trabajo final

Francisco Javier Pérez García
Manuel Ruiz Aguilar
Gabriel Francisco Sánchez Muñoz

Servidores Web de Altas Prestaciones, ICAR, ETSIIT, UGR

Introducción



docker

¿IA + Granja?

Fundamentos

- **Docker y Docker Compose:** Contenerización.
- **Ollama + Qwen3.5 0.8b:** Software para ejecutar modelos IA.
- **LiteLLM:** Balanceador para IAs.
- **Redis Stack:** Almacenamiento de caché semántico.

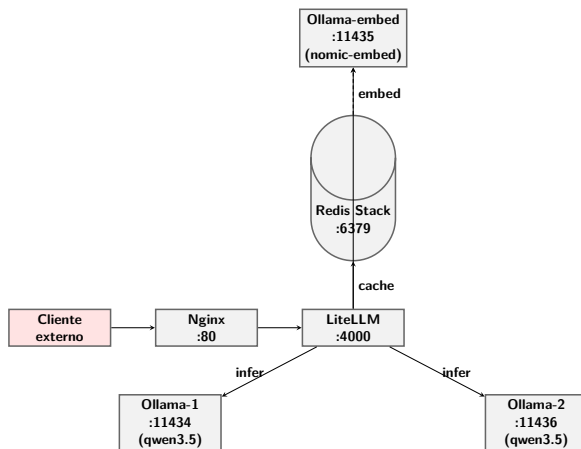
$$\text{sim}(\vec{a}, \vec{b}) = \frac{\vec{a} \cdot \vec{b}}{\|\vec{a}\| \cdot \|\vec{b}\|} \in [-1, 1]$$

- **Nginx:** Proxy inverso.
- **Locust y Python:** Test de funcionalidad.

Objetivos

- Granja IA organizada por funcionalidad y reproducible.
- Distribuir trabajo con balanceo de carga.
- Reducir la carga con consulta al caché semántico.
- Usar un proxy de alto rendimiento con respuestas en *streaming*.
- Realizar mediciones del rendimiento.

Arquitectura



Arquitectura del sistema: flujo de peticiones entre componentes.

Implementación

- **docker-compose.yml**: red *semantic-cache-net*, *depends_on*, puertos, comandos, variables de entorno, scripts, healthchecks y volúmenes.
- **configv2.yaml**: dos entradas para inferencia y una para embeddings, con sus debidos parámetros de APIs, el umbral de similitud a 0.85, tamaño de vector y tiempo de vida en caché.
- **nginx.conf**: redirige a LiteLLM:4000, permite *streaming* y extiende timeouts.

Análisis de rendimiento

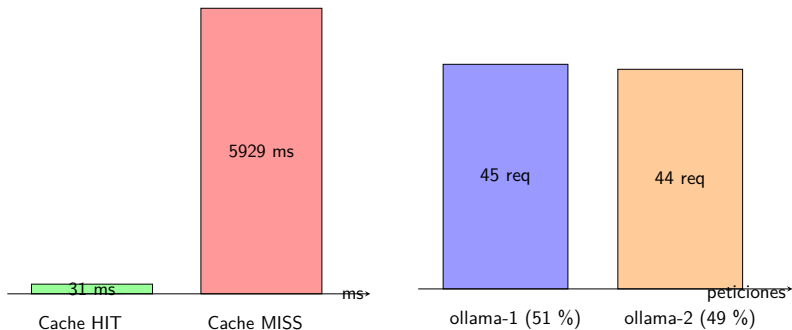
Resultados de test_litellm.py

HITs esperados (w) y obtenidos (r).

Grupo	Pet.	HITs (w)	HITs (r)	Acierto
Duplicados exactos	20	10	10	100 %
Paráfrasis Geografía	15	10	10	100 %
Paráfrasis Ciencia	15	10	4	40 %
Paráfrasis Historia	15	10	7	70 %
Relacionados distintos	9	0	0	—
No relacionados	5	0	1	—
Balanceo de carga	10	0	0	—

Análisis de rendimiento

Resultados de test_litellm.py



Factor de aceleración: 191,5×

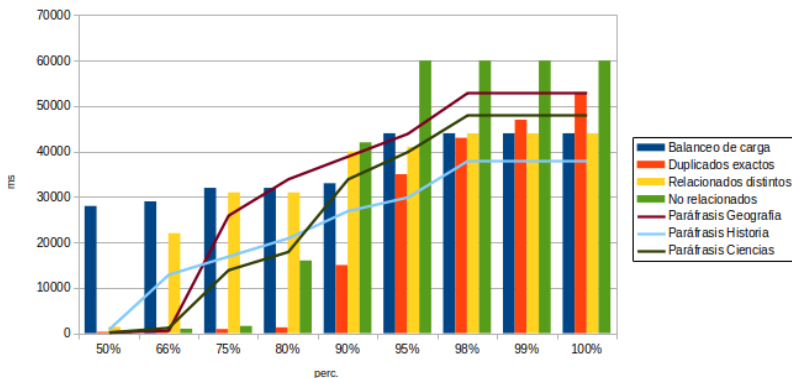
Análisis de rendimiento

Resultados de `locustfile.py`

Grupo	Pets.	50 %	Media	Mín.	Máx.	Av. Tam.
Duplic. exactos	129	310	3780	45	52933	308
Balanceo de carga	12	28000	25615	12583	44469	281
Paráfrasis Geo	24	250	9864	56	52635	304
Paráfrasis Hist	28	880	8357	59	37526	300
Paráfrasis Ciencia	40	250	7619	57	47966	303
Relac. distintos	23	1400	13803	60	44017	298
No relacionados	15	410	8302	60	59903	305

Análisis de rendimiento

Resultados de `locustfile.py`



Conclusiones

Cumplimiento de objetivos

Cumplido	Limitación
Aceleración de 191.5x en latencia con caché.	El umbral fijo no se adapta completamente.
Buena detección de paráfrasis en dominios con vocabulario diferenciado.	En ciencia; por ejemplo; no.
Balanceo equitativo.	Contamos solo con 2 réplicas de inferencia.
Cambios en la configuración independientes y sencillos.	La latencia se limita con la CPU y se aumenta con el paso intermedio de consultar caché.
Puede atender al menos dos peticiones por segundo.	—

Conclusiones

Trabajo futuro

- Umbral adaptativo.
- Caché multinivel.
- Más réplicas, con escalado dinámico.
- Aceleración por hardware.
- Monitorización con Prometheus y Grafana.
- Nuevo test de carga.

Red:

- Reglas según IP.
- Certificado SSL en Nginx.
- Optimización de envío según parámetros concretos.

FIN